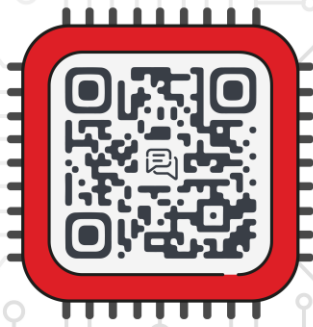




3

29

4350 sp

عملي

كلية الهندسة المعلوماتية

السنة الرابعة - الذكاء الصناعي

أساسيات تطبيق الشبكات العصبية باستخدام NumPy

عملي مشترك



RB Informatcs; 13/11/2025

محتوى مجاني غير مخصص للبيع التجاري

الشبكات العصبونية

إذا أردنا تطبيق شبكات عصبية بكفاءة عالية يجب أن نتقن التعامل مع المصفوفات متعددة الأبعاد باستخدام مكتبة Numpy .

السبب ؟ :

الشبكات العصبية في جوهرها عبارة عن سلسلة من العمليات الحسابية على أعداد ضخمة منظمة في مصفوفات (أو متجهات) بدلاً من كتابة حلقات تكرارية طويلة و بطيئة في بايثون تتيح لنا Numpy تنفيذ هذه العمليات المعقدة بسرعة وكفاءة عالية لأنها مكتوبة بلغات منخفضة المستوى مثل لغة C , ومحسنة لهذا النوع من الحسابات .

المصفوفات متعددة الأبعاد (multi dimensional arrays) :

المصفوفة متعددة الأبعاد هي ببساطة مجموعة من الأرقام منظمة في شكل معين يمكن أن تكون :

- بعد واحد (1D vector) صف من الأرقام تدعى متجه .
- بعدين (2D matrix) شبكة من الأرقام (تدعى مصفوفة) تشبه رقعة الشطرنج لها صفوف وأعمدة .
- ثلاثة أبعاد 3D أو أكثر (tensor) مكعب أو أشكال أكثر تعقيداً من الأرقام .

سنطبق الآن مصفوفة أحادية (ذات بعد واحد) حيث تدعى متجه vector :

```
1
2 import numpy as np
3 A = np.array([1, 2, 3, 4])
4 print(A)
5 print('dimensional of A is : ',np.ndim(A))
6 print('shape of A is : ',A.shape)
7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[1 2 3 4]
dimensional of A is : 1
shape of A is : (4,)
```

الشرح :

- تم إنشاء مصفوفة أحادية من أربعة عناصر
- للحصول على أبعاد المصفوفة نستخدم ndim تعطي عدد أبعاد المصفوفة وفي مثالنا لدينا بعد واحد .
- للحصول على شكل المصفوفة نستخدم shape حيث تعطي شكل المصفوفة على شكل tuple أي عدد العناصر في كل بعد (رقم , رقم) ولكن يوجد استثناء للمصفوفة الأحادية حيث تكون من الشكل (, رقم) (مهمة للتعبير أن المصفوفة من بعد واحد) .

لماذا هذا الشكل وليس فقط 4 ؟

لأن Numpy سواء كانت المصفوفة أحادية أو متعددة الأبعاد ستعيد دائماً مجموعة tuple هذا يسهل كتابة اكواد عامة حيث تعمل مع أي عدد من الأبعاد فمثلاً يمكنك دائماً الوصول للبعد الأول (أي بعد لا يشترط أن يكون الأول) من خلال A.shape[0] .
إذاً يمكننا القول أن shape هي مجموعة عناصر لإن النتيجة تكون بنفس الصيغة لكل أنواع المصفوفات .

مثال عن مصفوفة ثنائية البعد matrix :

```

1
2 import numpy as np
3 B = np.array([[1,2], [3,4], [5,6]])
4 print(B)
5 print('dimensional of B is : ',np.ndim(B))
6 print('shape of B is : ',B.shape)
7

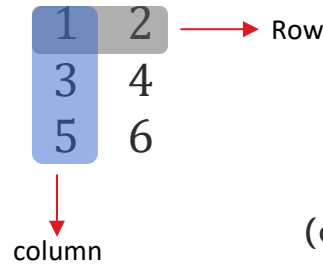
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
	[[1 2] [3 4] [5 6]] dimensional of B is : 2 shape of B is : (3, 2)			

الشرح :

تم إنشاء مصفوفة B (2D) من 3 صفوف وعمودان (3,2)
عدد أبعاد المصفوفة = 2 عن طريق التابع ndmin
شكل المصفوفة = (3,2) البعد الأول يشير إلى عدد الصفوف (3 عناصر)
البعد الثاني يشير إلى عدد الأعمدة (2عنصرين) .

- في المصفوفات ثنائية البعد التسلسل الأفقي يدعى row .
- التسلسل العمودي column .



الشكل : (3×2)

3 عناصر في البعد الأول (3 صفوف)
عنصرين في البعد الثاني (عمودين)
يبدأ ترقيم ل index في بايثون من الصفر .

سنحدث عن shape ل مصفوفة ذات ثلاث أبعاد التي تدعى tensor الشكل : (D, M, N)

يمكن التفكير فيها على أنها مكعب أو مجموعة من المصفوفات تحتوي على D مصفوفة وكل مصفوفة فيها بحجم $M \times N$

مثال :

مجموعة من الصور الملونة وكل صورة هي مصفوفة (ارتفاع $\times$ عرض) ومجموعة الصور هي Tensor (عرض $\times$ عدد الصور $\times$ ارتفاع)

إذاً shape هي أداة لفهم بنية البيانات قبل إجراء أي عملية حسابية أول شيء يجب التحقق منه هو shape المدخلات والمخرجات لضمان توافقهما .

ضرب المصفوفات matrix multiplication :

القاعدة الذهبية لضرب مصفوفتين A و B :

- عدد أعمدة المصفوفة الأولى A = عدد صفوف (أسطر) المصفوفة الثانية B
- $A \cdot B$
- $(m \times n) \cdot (n \times p) \rightarrow (m \times p)$
- النتج هو أسطر الأولى وأعمدة الثانية .

$$\begin{array}{|c|c|} \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} \cdot \begin{array}{|c|c|} \hline 5 & 6 \\ \hline 7 & 8 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 19 & 22 \\ \hline 43 & 50 \\ \hline \end{array}$$

A B

- يتم ضرب سطر بمجموعة أعمدة .
- للحصول على العنصر ذات index (0,0) من ضرب السطر الأول من المصفوفة A بالعمود الأول من المصفوفة B أي: $1 \times 5 + 2 \times 7$
- العنصر ذات index (0,1) ضرب السطر الأول من المصفوفة A بالعمود الثاني من المصفوفة B أي: $1 \times 6 + 2 \times 8$
- ونفس العملية على بقية العناصر حيث كل عنصر ينتج من ضرب سطر ب عمود .

- في Numpy نستخدم np.dot لتنفيذ عمليات الضرب الخطي حيث يتغير سلوكه بناءً على أبعاد المصفوفة .
- يمكن أن يضرب vector vector أو vector بمصفوفة والعكس أيضاً أو مصفوفة بمصفوفة .

مثال : تطبيق عملي لضرب مصفوفتان :

```

2 import numpy as np
3 A = np.array([[1,2], [3,4]])
4 print('shape of A is :',A.shape)
5 B = np.array([[5,6], [7,8]])
6 print('shape of B is :',B.shape)
7 x = np.dot(A, B)
8 print("result of A * B is : " , x)
9

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

shape of A is : (2, 2)
shape of B is : (2, 2)
result of A * B is : [[19 22]
[43 50]]

```

الشرح :

- لدينا A و B مصفوفتان ثنائية البعد (2,2) من سطرين وعمودين .
 أولاً يجب التحقق من الشرط (القاعدة الذهبية)
 في مثالنا : $shape (2 \times 2) dot (2 \times 2) = (2 \times 2)$
 أسطر الأولى وأعمدة الثانية
 بعد التحقق تم إجراء عملية الضرب من خلال np.dot والناتج هو مصفوفة (2×2) .

ملاحظة هامة :

- عملية ضرب المصفوفات ليست تبادلية أي $np.dot(A,B) \neq np.dot(B,A)$.
 مثل عملية الطرح أيضاً هي غير تبادلية $A - B \neq B - A$.

مثال :

```

2 import numpy as np
3 A = np.array([[1,2,3], [4,5,6]])
4 print('shape of A is :',A.shape)
5 B = np.array([[1,2], [3,4], [5,6]])
6 print('shape of B is :',B.shape)
7 x = np.dot(A, B)
8 print("result of A * B is : " , x)
9

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

shape of A is : (2, 3)
shape of B is : (3, 2)
result of A * B is : [[22 28]
[49 64]]

```

الشرح :

- لدينا المصفوفة A : (2,3) ثنائية الأبعاد من سطرين وثلاثة أعمدة
 لدينا المصفوفة B : (3,2) ثنائية الأبعاد من ثلاثة أسطر وعمودين

التحقق من الشكل :

Shape A dot B = C
 (2,3) (3,2) (2,2)

مثال آخر لمعرفة شكل مصفوفة الخرج :

Shape A dot B = C
 (3,2) (2,4) (3,4)

الناتج هو أسطر المصفوفة A و أعمدة المصفوفة B .

ضرب مصفوفة matrix ب vector :

Shape A dot B = C
 (3,2) 2 3

الشرط : أعمدة المصفوفة A = عدد عناصر المصفوفة B (vector)

والناتج هو vector عدد عناصره = عدد أسطر المصفوفة A هنا الناتج هو vector ب 3 عناصر أي [رقم , رقم , رقم]
 أي شيء يتم ضربه ب vector يكون الناتج هو vector .

ضرب vector ومصفوفة matrix

Shape A dot B = C
 2 (2,3) 3

الشرط : عدد عناصر ل vector = عدد أسطر المصفوفة B

والخرج هو vector عدد عناصره = عدد أعمدة المصفوفة B وفي مثالنا الخرج هو vector ب 3 عناصر
 [رقم , رقم , رقم] .

مثال :

```

1
2 import numpy as np
3 A = np.array([[1,2], [3, 4], [5,6]])
4 print('shape of A is :',A.shape)
5 B = np.array([7,8])
6 print('shape of B is :',B.shape)
7 x = np.dot(A, B)
8 print("result of A * B is : " , x)
9

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

shape of A is : (3, 2)
shape of B is : (2,)
result of A * B is : [23 53 83]

```

الشرح :

لدينا المصفوفة A هي 2D : (3,2)

والمتجه B (2,)

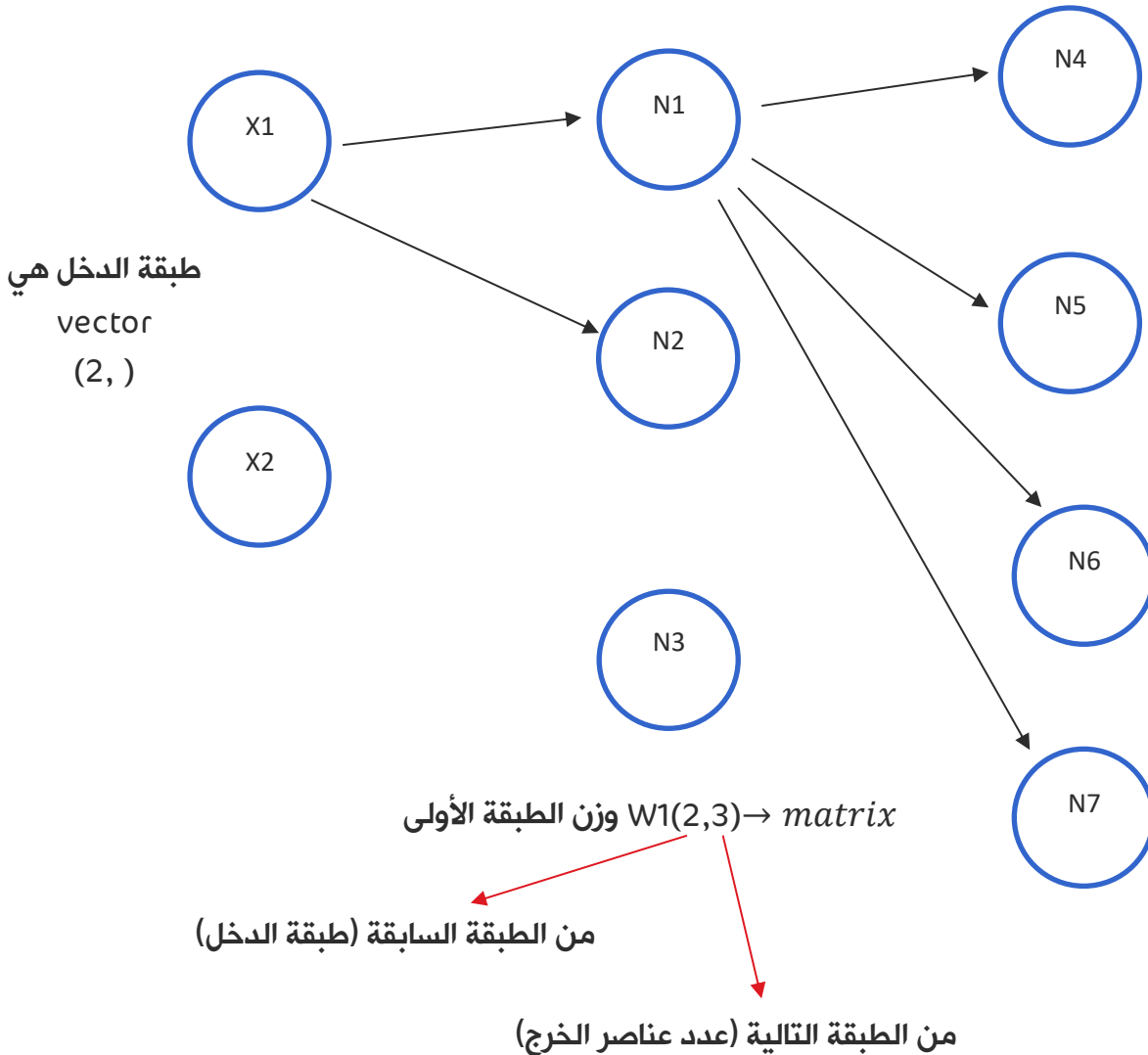
أولاً يجب معرفة شكل كل من A و B والتحقق من الشرط ثم إجراء عملية الضرب

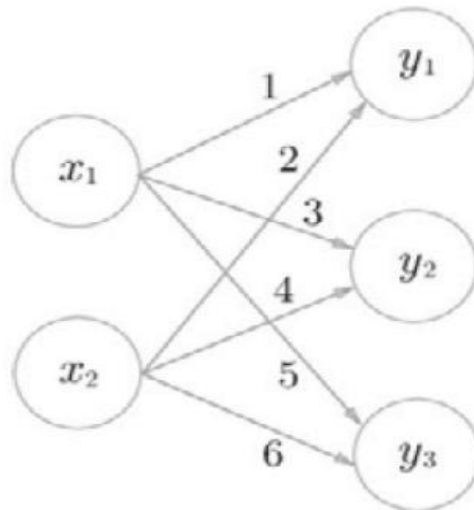
ضرب مصفوفات في الشبكات العصبية : neural network

بدلاً من كتابة حلقات تكرارية لحساب مخرج كل خلية عصبية على حدى نستخدم المصفوفات لحساب مخرجات طبقة عصبية بأكملها في خطوة واحدة .

لماذا نستخدمه :

- لأنه يمثل بشكل رياضي دقيق ومختصر ما يحدث في الطبقة العصبية .
- (كل خلية عصبية في الطبقة التالية تستقبل إشارة مرجحة من كل الخلايا العصبية في الطبقة الحالية) .
- لنفترض أن الشبكة العصبية تحتوي على أوزان فقط (بدون انحياز ودالة تنشيط)
- مثال بسيط لمعرفة شكل كل طبقة :





$$\text{Shape } x \cdot w = y$$

$\begin{matrix} 2 & (2,3) & 3 \end{matrix}$

يجب التأكد من shape كل من x, y, z حيث يجب أن يكون عدد العناصر في الأبعاد المقابلة ل x و w نفسه حيث x هي vector و w هي matrix والناتج هو vector .

لنفترض أننا ننتقل من طبقة تحتوي على M خلية عصبية إلى طبقة تحتوي على N خلية عصبية :

1. المدخلات x input vector :

تمثل إشارات الخلايا العصبية في الطبقة السابقة

شكلها $(M,)$ متجه من M عنصر .

2. مصفوفة الأوزان w weight matrix :

هذا هو الجزء الأهم , هذه المصفوفة التي تصل إلى الطبقة السابقة والطبقة التالية شكلها يجب أن يكون (M, N) لماذا؟

عدد الصفوف M يجب أن يتطابق مع عدد الخلايا في الطبقة السابقة (عدد عناصر x) لكي تتمكن من ضربها .
عدد الأعمدة N يجب أن يتطابق مع عدد الخلايا في الطبقة التالية التي نريد حساب مخرجاتها .

مثال :

```

1
2 import numpy as np
3 X = np.array([1, 2])
4 print('shape of X is :', X.shape)
5 W = np.array([[1, 3, 5], [2, 4, 6]])
6 print('shape of W is :', W.shape)
7 Y = np.dot(X, W)
8 print("result of X * W is : " , Y)
9

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

shape of X is : (2,)
shape of W is : (2, 3)
result of X * W is : [ 5 11 17]

```

الشرح :

لدينا المتجه x يحتوي على عنصرين (2,) ومصفوفة الأوزان (2,3) سطرين و 3 أعمدة
ملاحظة : لا ننسى التحقق من الشكل هذه أهم خطوة .

لنتحدث قليلاً عن مصفوفة الأوزان :

$$\begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{pmatrix}$$

- كل صف في w يمثل الأوزان القادمة من خلية عصبية واحدة في الطبقة السابقة .
- كل عمود في w يمثل مجموعة الأوزان المتجهة إلى خلية عصبية واحدة في الطبقة التالية .
- أي : السطر الأول : أوزان من أول دخل .
- السطر الثاني : أوزان من ثاني دخل .

سنتحدث الآن عن كيفية حساب الخرج y :

- حساب مخرج الخلية الأولى y_1 : نأخذ المتجه x ونضربه نقطياً في العمود الأول من w

$$y_1 = (x_1 \cdot w_{11}) + (x_2 \cdot w_{21})$$

- مخرج الخلية الثانية y_2 :

$$y_2 = (x_1 \cdot w_{12}) + (x_2 \cdot w_{22})$$

- مخرج الخلية الثالثة y_3 :

$$y_3 = (x_1 \cdot w_{13}) + (x_2 \cdot w_{23})$$

- فيكون الخرج النهائي هو vector (3,)

$$Y = [y_1, y_2, y_3]$$

- وبدلاً من كتابة 3 معادلات منفصلة أو حلقة تكرارية قمنا بكل هذا الحساب في سطر واحد :
 $Y = \text{np.dot}(X, w)$ هذا هو جوهر قوة الشبكات العصبونية و numpy معاً .

تطبيق شبكة عصبية من ثلاث طبقات Implementing a three-layer neural network

سنقوم الآن ببناء شبكة كاملة ونتبع البيانات خطوة بخطوة في رحلتها من المدخلات إلى المخرجات
(forward direction) .

قبل أن نكمل سنتحدث قليلاً عن مرورات الشبكة the Network passes :

للشبكة العصبية وصفان رئيسيان للعمل :

- المرور الأمامي forward للتنبؤ
- المرور الخلفي backward للتعلم

■ المرور الأمامي : مرحلة التنبؤ

■ الهدف : إعطاء إجابة أو تنبؤ بناءً على مدخلات معينة .

لتقريب الفكرة سنتحدث عن خط تجميع في مصنع :

المدخلات input : ندخل المواد الخام (البيانات مثل صورة رقم) .

الطبقات layers : تمر المواد الخام عبر عدة مراحل (الطبقات المخفية) .

■ في كل مرحلة يتم تحويلها خلال عملية : المجموع الموزون + الانحياز ثم دالة التنشيط .

المخرجات output في نهاية الخط نحصل على المنتج النهائي (التنبؤ) .

■ ببساطة :

المرور الأمامي هو استخدام الأوزان والانحيازات الحالية لحساب مخرجات الشبكة

في هذه المرحلة تكون الشبكة جاهزة وتعطي إجابة بناءً على ما تعلمته سابقاً .

■ متى نستخدمه :

عند التدريب : لمعرفة مدى خطأ تنبؤ الشبكة الحالي .

عند الاستدلال (inference) : بعد انتهاء التدريب نستخدم المرور الأمامي فقط للحصول على تنبؤات على

بيانات جديدة لم يرها النموذج من قبل .

■ المرور الخلفي : مرحلة التعلم

الهدف : تصحيح أخطاء الشبكة وتحسين أدائها للمستقبل

هذه العملية هي الأكثر تعقيداً وهي التي تمكن الشبكة من التعلم لا يمكن للشبكة أن تتعلم من المرور الأمامي وحده وإنما تحتاج إلى معرفة ما إذا كانت إجاباتها صحيحة أو خاطئة ومدى هذا الخطأ ومن المسؤول عن هذا الخطأ .

الخطوات :

1. حساب الخطأ calculate error :

عند المرور الأول نقارن مخرجات الشبكة (التنبؤ) والإجابة الصحيحة (المعروفة في مرحلة التدريب) الفرق بينهما هو

الخطأ أو الخسارة loss/Error

2. انتشار الخطأ للخلف propagate error backward :

هنا تكمن العبقرية , نأخذ هذا الخطأ ونبدأ في إرساله إلى الوراء من طبقة الخرج إلى الطبقات المخفية وطبقة الدخل

3. Assign Blame :

اثناء انتقال الخطأ للخلف يتم حساب مقدار مساهمة كل وزن وانحياز في هذا الخطأ الإجمالي هذا يعني أننا نحدد

الأوزان التي كانت مسؤولة عن الخطأ الأكبر

4. تحديث الأوزان update weights :

بناءً على مساهمة كل وزن في الخطأ نقوم بتعديل هذه الأوزان قليلاً في الاتجاه الذي يقلل الخطأ هذه العملية

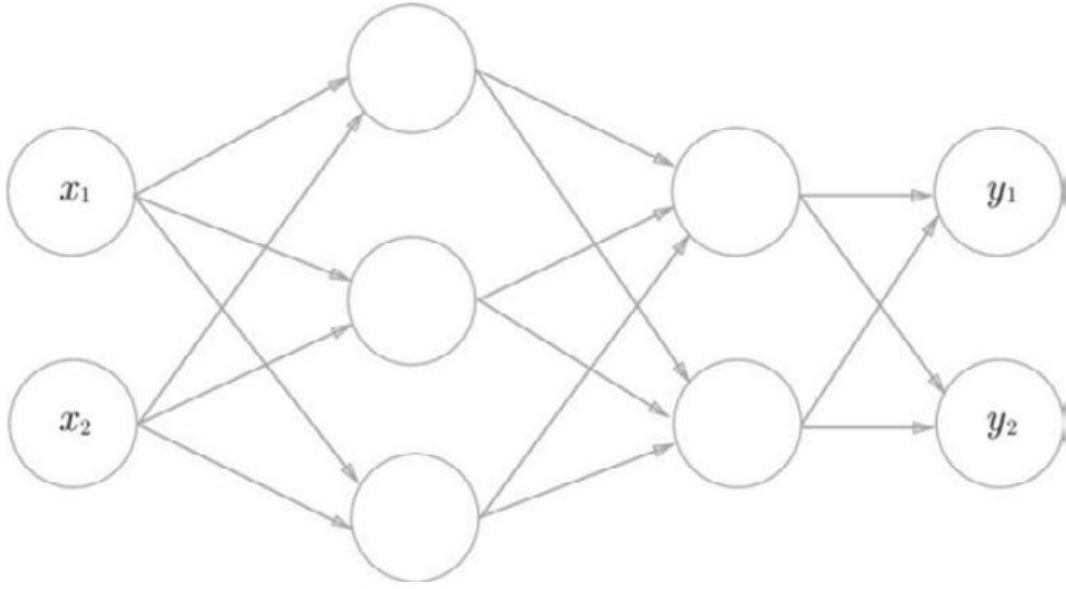
تسمى الانحدار التدريجي Gradient Descent نستخدم معدل تعلم (learning rate) لتحديد حجم هذه

التعديلات .

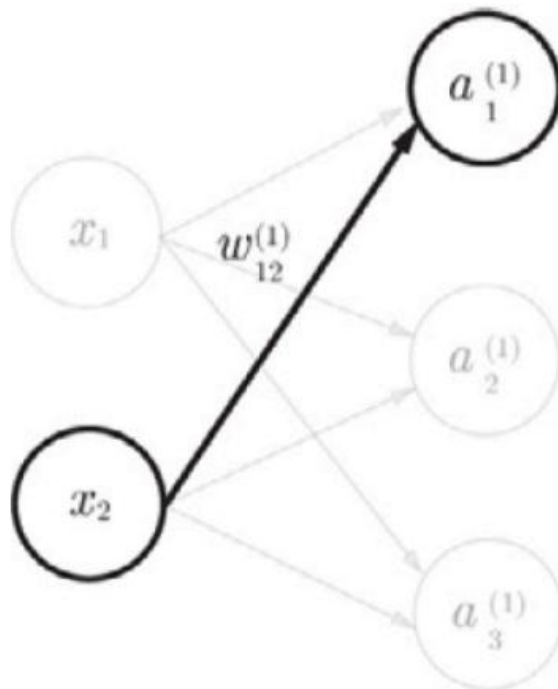
- ببساطة : المرور الخلفي هو معلّم يصحح إجابات الطالب (الشبكة) يرى الإجابة الخاطئة ويرجع خطوة بخطوة لتقول هذا الوزن كان كبيراً جداً وهذا الوزن كان صغيراً جداً هكذا يجب أن تعدلها للمرات القادمة .
- Forward : ماهي إجابتي بناءً على معرفتي الحالية .
- Backward : لقد أخطأت , كيف يمكنني تصحيح معرفتي لأكون أفضل في المرة القادمة .

والآن بالعودة إلى الفقرة :

الهيكل العام



المهم هو إمكانية حساب الشبكة العصبونية بشكل جماعي كحساب مصفوفة يمكن حساب كل طبقة في الشبكة العصبونية بشكل جماعي باستخدام ضرب المصفوفات لتتعارف على الرموز :



يشير إلى الطبقة

 w_{12}^1

خلايا الطبقة التالية والسابقة

يشير إلى أن هذا الوزن يربط الخلية العصبية الثانية x_2 في الطبقة السابقة إلى الخلية العصبية الأولى في الطبقة التالية حيث أن ترتيب الأرقام على النحو الآتي (الطبقة السابقة , الطبقة التالية) .

من أي خلية جاية $\rightarrow$ لأي خلية رايح $\leftarrow$

A^1 مصفوفة المجموعات الموزونة للطبقة الأولى .

Z^1 مخرجات الطبقة الأولى بعد تطبيق دالة التنشيط .

B^1 متجه انحياز الطبقة الأولى .

a_1^1 الخلية العصبية الأولى في الطبقة الأولى .

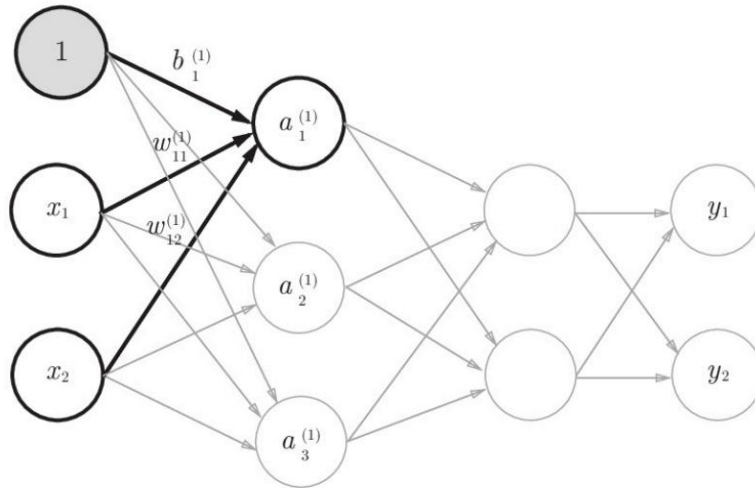
يجب التفريق بين الحرف الصغير والكبير

الصغير : من أجل عنصر واحد .

الكبير : من أجل أكثر من عنصر .

الآن سنقوم بتطبيق إشارة الانتقال بين الطبقات :

الخطوة 1 : من طبقة الدخول إلى الطبقة الأولى Hidden layer 1



▪ لا ننسى إضافة نيورون (خلية عصبية) إضافية في كل طبقة للتعبير عن الانحياز .

▪ الانحياز هو كما ذكرنا سابقاً مدى سهولة النيورون للإطلاق .

▪ في البايوس لدينا فقط رقم واحد من الأسفل يدل إلى أين يذهب .

▪ حيث عدد عناصر البايوس = عدد عناصر النيورونات الموجودة في الطبقة وهو عبارة عن vector

$$a_1^1 = w_{11}^1 x_1 + w_{12}^1 x_2 + b_1^1$$

- يمكننا التعبير عن أكثر من خلية من خلال :

مصفوفة المجموعات الموزونة للطبقة

$$A^1 = XW^1 + B^1 \rightarrow \text{الاولى} \leftarrow \text{الاولى}$$

$\swarrow$ $\searrow$
 مصفوفة اوزان الطبقة الاولى مدخلات الشبكة

- A^1 مصفوفة المجموعات الموزونة للطبقة الاولى
- مدخلات الشبكة X
- مصفوفة اوزان الطبقة الاولى W^1
- متجه انحياز الطبقة الاولى: B^1
- حيث أن :

$$X = (x_1, x_2) \rightarrow \text{vector}$$

$$\text{matrix} \leftarrow W^1 = \begin{pmatrix} W_{11}^1 & W_{21}^1 & W_{31}^1 \\ W_{12}^1 & W_{22}^1 & W_{32}^1 \end{pmatrix}$$

$$\text{vector} \leftarrow B^1 = (b_1^1, b_2^1, b_3^1)$$

$$A^1 = (a_1^1, a_2^1, a_3^1) \rightarrow \text{vector}$$

- ملاحظة : لا نضيف الانحياز إلى مصفوفة الأوزان لأننا بعد ضرب كل w ب x نقوم بجمع vector انحياز

```

2 import numpy as np
3 X = np.array([1.0, 0.5])
4 W1 = np.array([[0.1, 0.3, 0.5], [0.2, 0.4, 0.6]])
5 B1 = np.array([0.1, 0.2, 0.3])
6 print('shape of W1 is :', W1.shape)
7 print('shape of X is :', X.shape)
8 print('shape of B is :', B1.shape)
9 A1 = np.dot(X, W1) + B1
10 print(A1)
11

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

shape of W1 is : (2, 3)
shape of X is : (2,)
shape of B is : (3,)
[0.3 0.7 1.1]

```

- الشرح:

لدينا المتجه x مكون من عنصرين (2,).
 والمصفوفة $w1$ هي (2,3) مكونة من سطرين و 3 أعمدة .
 نتحقق من الشرط وهو عدد عناصر المتجه يجب أن يساوي عدد أسطر المصفوفة $w1$ و هنا نتحقق من الشرط وهو عدد عناصر المتجه فيكون الخرج لدينا هو متجه من 3 عناصر ثم نضيف متجه الانحياز بعد حساب المجموع الموزون و الانحياز في الطبقة الاولى تأتي الخطوة التالية :

الخطوة 2 : تطبيق دالة التنشيط

يتم تطبيق $h()$ دالة التنشيط على الخرج a فينتج z الخرج بعد تطبيق دالة التنشيط ويكون z هو دخل الطبقة التالية إذاً : $A1$ تمر عبر دالة تنشيط sigmoid بدونها ستكون الشبكة العصبية مجرد نموذج خطي بسيط .

```

8
9 def sigmoid(x):
10     return 1 / (1 + np.exp(-x))
11
12 Z1 = sigmoid(A1)
13 print(Z1)
14

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

[0.57444252 0.66818777 0.75026011]

الخطوة 3 : من الطبقة الأولى إلى الطبقة الثانية Hidden layer 2

نكرر نفس العملية حيث $z1$ مخرجات الطبقة الأولى هي مدخلات الطبقة الثانية

```

15 W2 = np.array([[0.1, 0.4], [0.2, 0.5], [0.3, 0.6]])
16 B2 = np.array([0.1, 0.2])
17 print('shape of Z1 is :',Z1.shape)
18 print('shape of W2 is :',W2.shape)
19 print('shape of B2 is :',B2.shape)
20 A2 = np.dot(Z1, W2) + B2
21 Z2 = sigmoid(A2)
22 print(Z2)
23

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

shape of Z1 is : (3,)
 shape of W2 is : (3, 2)
 shape of B2 is : (2,)
 [0.62624937 0.7710107]

الخطوة 4 : من الطبقة الثانية إلى طبقة الإخراج output layer

العملية متشابهة ولكن دالة التنشيط في طبقة الإخراج قد تكون مختلفة حسب نوع المشكلة .

```

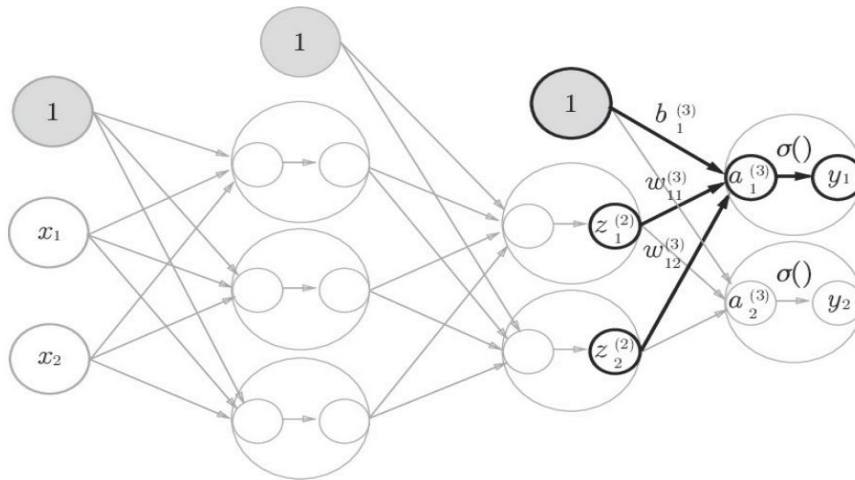
24 def identity_function(x):
25     return x
26 W3 = np.array([[0.1, 0.3], [0.2, 0.4]])
27 B3 = np.array([0.1, 0.2])
28 A3 = np.dot(Z2, W3) + B3
29 print('shape of Z2 is :',Z2.shape)
30 print('shape of W3 is :',W3.shape)
31 print('shape of B3 is :',B3.shape)
32 Y = identity_function(A3)
33 print(Y)
34

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

shape of Z2 is : (2,)
 shape of W3 is : (2, 2)
 shape of B3 is : (2,)
 [0.31682708 0.69627909]

لقد استخدمنا التابع identity function لانها تعيد المدخلات كما هي .



تنظيم الكود : يمكننا تنظيم الكود في دالتين :

- تهيئة الشبكة العصبية `init_network`
- عملية الانتشار الأمامي `forward`

تصميم طبقة المخرجات Designing the output layer

اختيار دالة التنشيط لطبقة الخرج يعتمد على نوع المشكلة التي تحلها الشبكة :

1. مشاكل الانحدار Regression problem :

هي مشاكل هدفنا فيها التنبؤ برقم (قيمة رقمية مستمرة) وليس اختيار تصنيف (مثل مزعج/غير مزعج , قطرة/كلب) أمثلة :

- التنبؤ بسعر منزل قد يكون 150000 أو 356000 .
- التنبؤ بدرجة حرارة قد تكون 35 درجة مئوية .
- التنبؤ بمبيعات متجر بشهر محدد .

الدالة المستخدمة في طبقة الخرج identity function :

لماذا ؟

لأنها أبسط دالة $f(x)=x$ لا تغير أي شيء

السبب :

في مسائل الانحدار نريد أن تكون مخرجات الشبكة هي القيمة المتوقعة مباشرة , إذا كانت الشبكة هي تنبؤ بأن سعر المنزل هو 150000 فنحن نريد أن تكون المخرجات النهائية 150000 بالضبط وأي دالة أخرى مثل sigmoid ستقوم بضغط هذه القيمة إلى نطاق معين (مثل 0 إلى 1) مما يجعلها عديمة الفائدة للتنبؤ بالقيم الأصلية .

2. مسائل التصنيف classification function :

هي مسائل نريد فيها تصنيف المدخل إلى واحد من فئات محددة ومعلومة مسبقاً
يوجد نوعان :

1. تصنيف ثنائي binary classification
2. تصنيف متعدد الفئات multi_class classification

التصنيف الثنائي : فئتان فقط (نعم/لا , صح / خطأ)

الدالة المستخدمة sigmoid

لماذا؟ لان دالة sigmoid تضغط أي قيمة إلى نطاق بين 0 و 1 هذا النطاق مثالي لتمثيل الاحتمالية
يمكننا تفسير المخرج على أنه احتمالية انتماء المدخل إلى الفئة الأولى .

إذا كان المخرج $0.5 <$ نختار الفئة الأولى .

إذا كان المخرج $0.5 \geq$ نختار الفئة الثانية .

مثال : في مشكلة البريد المزعج إذا كانت مخرجات sigmoid هي 0.95 فنحن على ثقة عالية بأن البريد مزعج .

التصنيف متعدد الفئات :

أكثر من فئتين مثل التعرف على الأرقام من $0 \leftarrow 9$ أو تصنيف صور إلى قطة /كلب / طائر

الدالة المستخدمة soft max

لأنها مهمة خصيصاً لهذا النوع من المشاكل تأخذ مجموعة من الدرجات الخام logits وتحوله إلى توزيع احتمالي .

- خاصية 1 : كل مخرجاتها بين 0 و 1
- خاصية 2 : مجموع كل المخرجات يساوي 1 بالضبط
- خاصية 3 : الخلية العصبية التي لديها أعلى قيمة a_k ستكون لها أعلى احتمالية في y_k

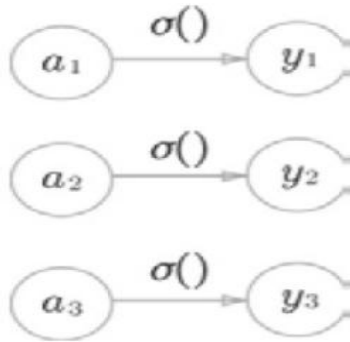
التفسير : إذا كان لدينا 10 فئات الأرقام من $0 \leftarrow 9$ فإن مخرجات الدالة ستكون 10 قيم

القيمة الأولى هي احتمالية أن يكون الرقم هو 0 والثانية احتمالية أن يكون الرقم 1 وهكذا ,
الفئة ذات أعلى احتمالية هي توقع الشبكة .



السبب	دالة التنشيط طبقة الخرج	مثال	الهدف	نوع المشكلة
للحصول على القيمة المتوقعة مباشرة دون تغيير	Identity	سعر منزل	التنبؤ بقيمة مستمرة	الانحدار
للحصول على احتمالية بين 0 و 1 لفئة واحدة	Sigmoid	بريد مزج	الاختيار بين فئتين	التصنيف الثنائي
للحصول على توزيع احتمالي على جميع الفئات (مجموعها=1)	softmax	التعرف على رقم	الاختيار من عدة فئات	التصنيف متعدد الفئات

تمثيل الشبكة العصبية باستخدام identity :



■ قانون الدالة softmax :

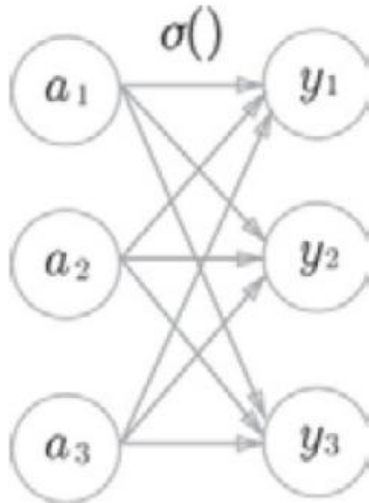
$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$

. a_k إشارة الإدخال للخلية k .

. K: مخرج الخلية العصبية .

. $\sum_{i=1}^n \exp(a_i)$: مخرج الدالة الأسية لجميع مخرجات الطبقة .

تمثيل الشبكة العصبونية باستخدام softmax :



حيث يرتبط خرج الدالة لجميع إشارات الدخل بسهم وتتأثر لكل خلية عصبية من الخرج لجميع إشارات الدخل

التطبيق : توضيح آلية حساب الدالة

```

1
2 import numpy as np
3 a = np.array([0.3, 2.9, 4.0])
4 exp_a = np.exp(a) # Exponential function
5 print(exp_a)
6 sum_exp_a = np.sum(exp_a) # Sum of exponential functions
7 print(sum_exp_a)
8 y = exp_a / sum_exp_a
9 print(y)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

[ 1.34985881 18.17414537 54.59815003]
74.1221542101633
[0.01821127 0.24519181 0.73659691]

```

فيكون الشكل النهائي للدالة :

```

def softmax(a):
    exp_a = np.exp(a)
    sum_exp_a = np.sum(exp_a)
    y = exp_a / sum_exp_a
    return y

```

■ Overflow problem :

عندما تكون قيم a_k كبيرة جداً فإن $\exp(a_k)$ يكبر بشكل كبير جداً والحاسب لا يستطيع التخزين وتحدث مشكلة overflow وتعطي خرج int .

الحل: يخل القيم إلى مجال ثاني بحيث يعطي نفس القيمة الاحتمالية أي لا يتغير الخرج
نطرح القيمة القصوى من المدخلات قبل حساب الأس .
سنوضح من خلال المثال المشكلة والحل :

```

2 import numpy as np
3 a = np.array([1010, 1000, 990])
4 y=np.exp(a) / np.sum(np.exp(a))
5 print(y)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

c:\vs_projects\python_project\pdf3.py:4: RuntimeWarning: overflow encountered in exp
y=np.exp(a) / np.sum(np.exp(a))
c:\vs_projects\python_project\pdf3.py:4: RuntimeWarning: invalid value encountered in divide
y=np.exp(a) / np.sum(np.exp(a))
[nan nan nan]

```

الحل:

```

2 import numpy as np
3 a = np.array([1010, 1000, 990])
4 c = np.max(a)
5 print(a-c)
6 y=np.exp(a - c) / np.sum(np.exp(a - c))
7 print(y)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

[ 0 -10 -20]
[9.99954600e-01 4.53978686e-05 2.06106005e-09]

```

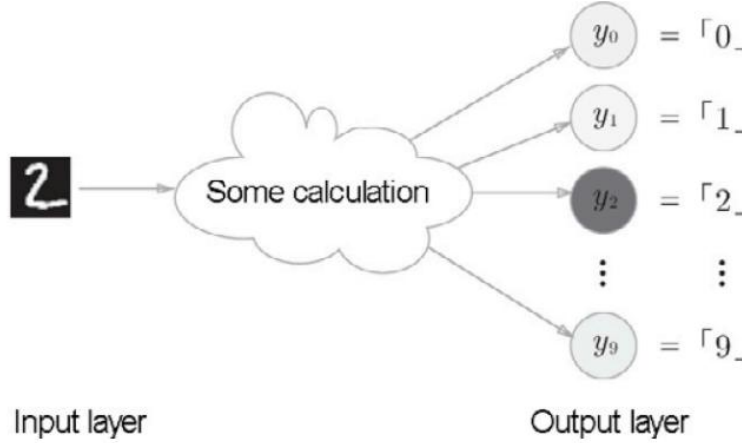
أولاً نأخذ أعلى قيمة عن طريق `np.max` ثم اسندنا القيمة إلى المتغير `c` أعلى قيمة في المتجه `a` ثم طرح هذه القيمة من جميع القيم في المتجه .
في `a-c` تكون أعلى قيمة في `a` قيمتها في المجال الجديد `a-c` تساوي الصفر وباقي القيم قريبة من الصفر .



■ عدد الخلايا في طبقة الخرج :

يجب تحديد عدد الخلايا العصبية في طبقة الإخراج حسب الحاجة وذلك حسب المشكلة المراد حلها , في مسائل التصنيف يستخدم عادةً عدد الفئات المطلوب تصنيفها كعدد الخلايا العصبية في طبقة الإخراج .

مثال : للتنبؤ برقم من 0 إلى 9 من صورة إدخال (تصنيف من 10 فئات) توضع 10 خلايا عصبية في طبقة الإخراج .



شرح الصورة :

- يوضح الرسم كيف يتم تمثيل و تفسير مخرجات الشبكة العصبية لمشكلة تصنيف الأرقام من 0 إلى 9
- طبقة الإدخال : يتم إدخال صورة لرقم مكتوب بخط اليد وهو 2
- بعض العمليات الحسابية: يمثل هذا الجزء جميع الطبقات المخفية وعمليات الضرب النقطي مع الأوزان وتطبيق دوال التنشيط التي تحدث داخل الشبكة العصبية .
- طبقة الإخراج : تتكون من 10 خلايا عصبية كل خلية تمثل رقم أو فئة تصنيف محددة .
- الخلية العلوية y_0 تمثل الرقم 0 , الخلية y_1 تمثل الرقم 1 إلى أن نصل إلى الخلية y_9 التي تمثل الرقم 9 .

قراءة قيم الإخراج :

- الألوان الرمادية: تمثل درجات الظل المختلفة للرمادي قيمة أو احتمالية الخلية العصبية في طبقة الإخراج .
- حيث تشير القيمة إلى مدى ثقة الشبكة بأن الإدخال ينتمي إلى الفئة المقابلة لهذه الخلية .
- التنبؤ النهائي prediction يتم اتخاذ القرار بناءً على الخلية العصبية التي تخرج القيمة الأكبر .
- في مثالنا الخلية العصبية y_2 المقابلة للرقم 2 هي الأغصق لوناً مما يعني أنها أخرجت أكبر قيمة بين الخلايا العشرة .
- هذا يدل على أن الشبكة العصبية تتوقع أن الإدخال ينتمي إلى الفئة التي تقابل y_2 أي أن التنبؤ هو الرقم 2 .

التعرف على الأرقام المكتوبة بخط اليد : (تطبيق عملي على كل ما سبق)

(a) مجموعة البيانات MNIST Dataset :

تحتوي على 60000 صورة للتدريب و 10000 صورة للاختبار

كل صورة هي رقم مكتوب بخط اليد من 0 إلى 9

كل صورة هي بحجم 28×28 بكسل وكل بكسل له قيمة بين أسود و 255 أبيض .

(b) تجهيز البيانات :

يتم تطبيق Flatten على كل صورة لتصبح متجه واحد من 784 عنصر ($28 \times 28 = 784$)

هذا المتجه سيكون مدخل الشبكة العصبية .

(c) بناء الشبكة والاستدلال : inference :

- طبقة الدخل : 784: خلية عصبية (واحدة لكل بكسل) من حجم الصورة (28×28) .
 - طبقة الخرج : 10 خلايا عصبية (واحدة لكل رقم من 0 إلى 9) .
 - الطبقات المخفية : مثلاً 50 خلية في الطبقة الأولى و 100 في الثانية (يمكن تغيير هذه الأرقام) .
- (d) قياس الدقة accuracy :

- نحرر كل صورة من مجموعة الاختبار عبر الشبكة (predict) .
- نستخدم np.argmax للحصول على فهرس أعلى احتمالية في مخرجات softmax .
- هذا الفهرس هو توقع الشبكة للرقم .
- نقارن التنبؤ بالعلامة الحقيقية .
- الدقة = عدد التنبؤات الصحيحة / إجمالي عدد الصور .



```

1
2 from sklearn.datasets import fetch_openml
3 from sklearn.model_selection import train_test_split
4 import numpy as np
5
6 X, y = fetch_openml('mnist_784', return_X_y=True, as_frame = False)
7 x_train, x_test, t_train, t_test = train_test_split(X, y, test_size=10000)
8 t_train = t_train.astype(int)
9 t_test = t_test.astype(int)
10 print('shape of x_train', x_train.shape)
11 print('shape of t_train', t_train.shape)
12 print('value of t_train[0]', t_train[0])
13

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

shape of x_train (60000, 784)
shape of t_train (60000,)
value of t_train[0] 3

```

الشرح :

- مكتبة fetch_openml تستخدم لتمثيل مجموعة بيانات كبيرة
- مكتبة train-test-split تقسيم البيانات بشكل عشوائي إلى بيانات تدريب وبيانات اختبار
- مكتبة one Hot Encoder أداة لتحويل البيانات التصنيفية (مثل الأرقام من 0 إلى 9) إلى صيغة أخرى مفيدة للشبكات العصبية

شرح السطر : `x,y=fetch-openml('mnist 784', return x y=true,as frame = False)`

تبدأ عملية تحميل البيانات حيث 'mist-784' اسم مجموعة بيانات MNIST التي تحتوي على 784 عمود (بكسل) `return x-y` تعيد النتيجة على شكل متغيرين .

`x`: يمثل بيانات الصور (الميزات)

`Y`: يمثل التصنيفات (labels)

`as-frame` يطلب إرجاع البيانات كمصفوفات بدلاً من إطار بيانات `DataFrame` من مكتبة `pandas` وهذا هو الشكل الأنسب للتعامل مع الشبكات العصبية .

`x`: مصفوفة ضخمة تحتوي على كل صور الأرقام .

`Y`: يحتوي على الرقم المقابل لكل صورة في المتجه `X` .

تقسيم البيانات إلى تدريب واختبار

لماذا نقسم البيانات؟

تخيل أنك ستدخل امتحان هل من الحكمة أن تدرس فقط من أسئلة الامتحان النهائي نفسها ؟

بالطبع لا , لأنك ستحفظ الإجابات دون أن تفهم الأفضل هو أن تدرس من أسئلة تدريب (مجموعة كبيرة) ثم تقوم بحل الامتحان التجريبي (مجموعة أصغر)

لم تر أسئلة من قبل لتقييم فهمك الحقيقي , هذا تماماً ما نفعله هنا .

بيانات التدريب :

t-train , x-train

هي كتاب الدراسة سنعطيه للشبكة العصبية لترى منا الآف الأمثلة وتتعلم كيفية التعرف على الأرقام

بيانات الاختبار:

x-test, t-test هي الامتحان النهائي نعطيها للشبكة العصبية بعد أن تنتهي من التدريب نرى مدى قدرتها على التعرف على أرقام لم تكن قد رأتها من قبل هذا يعطينا تقيماً حقيقياً وموثوقاً لأدائها .

Train-test-split(x,y---) تقوم بتقسيم بيانات المدخلات x والتسميات y بالتوازي
test-size=10000 يحدد حجم مجموعة الاختبار لتكون 10000 صورة من أصل 70000

الناتج :

x-train المدخلات للتدريب 60000 صورة / x-test المدخلات للاختبار 10000

t-train التسميات للتدريب 60000 تسمية / t-test التسميات للاختبار 10000

تطبيق وتحويل الإجابات labels:

t-train=t-train.astype(int) t-test=-----

عندما يتم يتم تحميل البيانات تكون الإجابات (y) على شكل نصوص strings مثل : '2' , '6' .
الشبكات العصبية والعمليات الحسابية لا تفهم النصوص بل تتعامل مع الأرقام فقط، إذاً
قمنا بتحويل كل التسميات من نص إلى أرقام صحيحة int الآن كل من t-train , t-test
يحتويان على الأرقام مثل 2,6,7 .

شكل Dataset (70000,784)

عندي 70000 صورة وكل صورة فيها 784 قيمة

(60000,784) مصفوفة التدريب تحتوي على 60000 صف (كل صف يمثل صورة) و 784 عمود
(كل عمود يمثل بكسل) .

(, 60000) متجه الإجابات يحتوي على 60000 عنصر حيث كل عنصر هو الرقم المقابل للصورة .
t-train[0] أول صورة في مجموعة التدريب (الخرج صورة للرقم 6) .
لو كان [0][0] أول ميزة (feature) + أول صورة .



عرض الصورة :

```

2 from sklearn.datasets import fetch_openml
3 from sklearn.model_selection import train_test_split
4 import numpy as np
5 from PIL import Image
6 import matplotlib.pyplot as plt
7
8 X, y = fetch_openml('mnist_784', return_X_y=True, as_frame = False)
9 x_train, x_test, t_train, t_test = train_test_split(X, y, test_size=10000)
10 t_train = t_train.astype(int)
11 t_test = t_test.astype(int)
12
13 def img_show(img):
14     pil_img = Image.fromarray(np.uint8(img))
15     plt.imshow(img, cmap='gray')
16     plt.show()
17
18 img = x_train[10]
19 label = t_train[10]
20 print('label of t_train[10]', label)
21 print('shape of img : ', img.shape)
22 # Reshape the image based on the original size
23 img = img.reshape(28, 28)
24 #after reshape
25 print('shape of img : ', img.shape) # (28, 28)

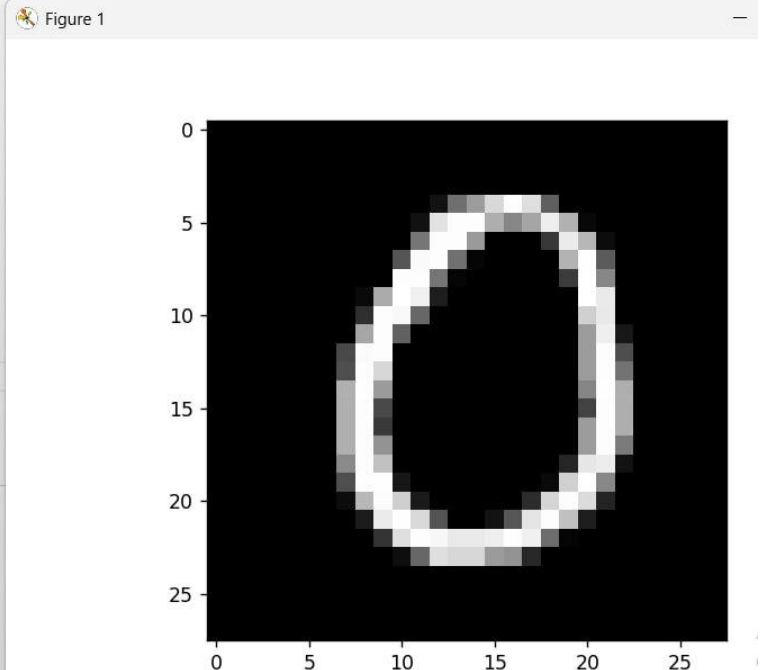
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

label of t_train[10] 0
shape of img : (784,)
shape of img : (28, 28)

```



▪ مكتبة (python imaging library) PIL :

تستخدم لمعالجة الصور وإنشائها وحفظها وعرضها .

▪ دالة عرض الصورة (img-show) :

هذه الدالة هي المسؤولة عن تحويل مصفوفة Numpy إلى صيغة يمكن عرضها على الشاشة .

▪ def :

تعريف الدالة التي تقبل كمدخل مصفوفة ثنائية الأبعاد 28×28 تمثل الصورة img .

▪ Pil-img :

(img) np.uint8 يتم تحويل نوع بيانات المصفوفة إلى عدد صحيح غير سالب بحجم 8 بت ,

(unit8) هذا النوع ضروري لأن قيم البكسل تتراوح من 0 إلى 255 ,

وهو ما يتوافق تماماً مع بت هذا يضمن التعامل الصحيح مع بيانات الصورة .

▪ `plt.imshow(img)` :

تأمر matplotlib بعرض مصفوفة `img` كصورة .

▪ `camp='gray'` :

يحدد أن الصورة يجب أن تعرض بألوان رمادية نظراً لأن صور MNIST هي صور رمادية فإن هذا يضمن عرضها باللون الأسود والأبيض ودرجات الرمادي بينهما بدلاً من الألوان الافتراضية .

▪ `plt.show()` :

عرض الصورة .

▪ `Img=x-train[10]` :

تم اجتياز الصورة ذات الفهرس 10 من مجموعة المدخلات للتدريب .

▪ `Label =`

يتم اختيار التسمية المقابلة (الرقم الحقيقي) لنفس العينة من مجموعة التسميات للتدريب

▪ `Img.shape` :

شكل مصفوفة الصورة قبل التعديل والناتج هو (784) هذا يؤكد أن الصورة يتم تحميلها وهي حالياً متجه

▪ `Img.reshape` (إعادة التشكيل) :

هذه الخطوة الأهم للعرض بما أن الدالة `img-show` تتوقع مصفوفة ثنائية الأبعاد 28×28

يتم استخدام الدالة `reshape` لتحويل المتجه (784) إلى مصفوفة ثنائية الأبعاد 28 صف و 28 عمود

▪ `Img-show` :

عرض الصورة على الشاشة

▪ تنفيذ الكود لفكرة الاستدلال والدقة :

```

23 def init_network():
24     np.random.seed(42)
25     network = {
26         'W1': np.random.randn(784, 50) * 0.01,
27         'b1': np.zeros(50),
28         'W2': np.random.randn(50, 100) * 0.01,
29         'b2': np.zeros(100),
30         'W3': np.random.randn(100, 10) * 0.01,
31         'b3': np.zeros(10)
32     }
33     return network
34 def predict(network, x):
35     pass
36 x, t = x_test, t_test
37 network = init_network()
38 accuracy_cnt = 0
39 for i in range(len(x)):
40     y = predict(network, x[i])
41     p = np.argmax(y) # Obtain the index of the most probable element
42     if p == t[i]:
43         accuracy_cnt += 1
44 print("Accuracy:" + str(float(accuracy_cnt) / len(x)))
45

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Accuracy:0.1008

▪ الدالة `init_network` :

مسؤولة عن إنشاء هيكل الشبكة العصبونية وتعيين القيم الأولية (الأوزان والانحيازات) لجميع الطبقات

▪ `random.randn` :

تعطي قيم عشوائية ضمن مصفوفة وتكون القيم `float` مسحوبة من توزيع احتمالي

▪ `W1` :

مصفوفة أوزان بحجم (784,50) , 784 مدخلات الصورة 28×28 فيوتشرات و 50 هي عدد الخلايا العصبية في الطبقة الأولى

0.01 * لضمان قيم ابتدائية صغيرة .

▪ `b1` :

متجه الانحياز بحجم (50,) وكل قيمه صفرية

وهكذا لبقية الطبقات حيث طبقة الإخراج هي 10 (لتمثيل الأرقام من 0 إلى 9) .

▪ دالة التنبؤ `predict` :

مسؤولة عن تنفيذ عملية الإرسال الأمامي `forward propagation` .

▪ **تقييم دقة الشبكة :**

▪ `X,t= x_test , t_test` : تعيين البيانات

▪ `network= init_network()` : تهيئة الشبكة

▪ `Accuracy_cnt` : عداد الدقة

▪ `for i in range(len(x))` : حلقة التقييم

▪ `p= np.argmax(y)` : الحصول على التنبؤ

▪ `if p==t[i]` : التحقق من الدقة

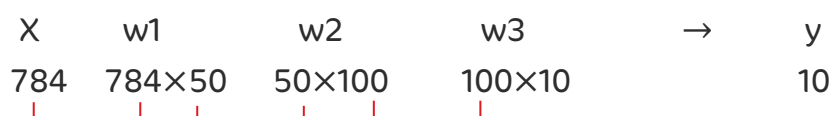
▪ `accuracy-cnt+=1` : زيادة العداد

▪ `print()` : حساب وعرض الدقة

ملاحظة :

نظراً لأن الأوزان تم تهيئتها عشوائياً ولم يتم تدريبها أبداً (باستخدام الانتشار الخلفي) فإن دقة هذه الشبكة ستكون منخفضة جداً من بين 10 فئات .

حيث تم التحقق من `shape`



شرح مفهوم الدفعة (Batch Processing)

■ ما هي الدفعة (The Concept of a Batch) ؟

الدفعة هي عبارة عن مجموعة مُنظمة من بيانات الإدخال يتم تجميعها ومعالجتها معاً في خطوة واحدة، بدلاً من معالجة كل عينة بشكل منفصل.

التعريف: "مجموعة مُنظمة من بيانات الإدخال (An organized set of input data) "يتم إرسالها إلى الشبكة العصبونية في عملية واحدة.

الهدف: بدلاً من إرسال صورة MNIST واحدة بحجم (784,) في كل مرة، يتم تجميع 100 صورة معاً في مصفوفة واحدة بحجم (100,784) وإرسالها إلى الشبكة (stack of images)

أي : أصبح لدينا الدخل مصفوفة بدلاً من متجه : نقوم بتجميع عدة مدخلات في مصفوفة وتمريرها الى الشبكة .
فيكون لدينا شكل كل من :

■ شكل مدخلات الشبكة : (batch_size , 784)

■ شكل الخرج : (batch_size , 10)

مثال :

إذا كان لدينا حجم الدفعة هو 100 صورة :

X_batch.shape : (100,784)

Y_batch.shape : predict(network , x_batch) وهذا سيعيد لنا مصفوفة (100,10)

حيث:

Y_batch[0] : هذا يعيد لنا توقع الصورة الاولى

Y_batch [1] : هذا يعيد لنا توقع الصورة الثانية وهكذا

■ الميزة الكبرى: الكفاءة الحسابية (Computational Efficiency) :

يكن السبب الرئيسي لاستخدام الدفعات في كونها تزيد من سرعة وكفاءة الحسابات بشكل كبير في التعلم الآلي.

1. تحسين مكتبات الحساب العددي (NumPy Optimization)

مكتبات الحساب العددي مثل NumPy مُحسّنة بشكل خاص للقيام بالعمليات على المصفوفات والمتجهات الكبيرة (Vectorization).

عند استخدام ضرب المصفوفات (Matrix Multiplication) لحساب الطبقة. $Y = XW + B$:

في المعالجة المتسلسلة x : هو متجه (784,)، والعملية تكون أبطأ.

في معالجة الدفعة x : هي مصفوفة (N* 784) ، والعملية تصبح ضرب مصفوفة في مصفوفة، مما يسمح لوحداث المعالجة المركزية (CPU) ووحدات معالجة الرسومات (GPU) بتطبيق الحسابات بالتوازي وبكفاءة أعلى بكثير.

وبالتالي، بدلاً من تكرار عملية حساب (1 * 784) صورة 100 مرة، يتم إجراء عملية حساب (100 * 784) مصفوفة مرة واحدة، مما يقلل بشكل كبير من وقت المعالجة لكل صورة .

2. تقليل الحمل على ناقل البيانات (Reducing Bus Band Load)

ناقل البيانات (Bus Band) هو المسار الذي يتم من خلاله نقل البيانات بين ذاكرة النظام (RAM) ووحدة المعالجة المركزية (CPU) أو وحدة معالجة الرسومات (GPU).

الحل: معالجة الدفعة تسمح بتحميل الأوزان ($Matrices W_1, W_2, W_3$) في الذاكرة مرة واحدة فقط، واستخدامها لحساب مخرجات جميع الصور الـ 100 الموجودة في الدفعة. هذا يقلل من حمل النقل على ناقل البيانات (Bus Band Load) مقارنة بالاضطرار إلى قراءة وكتابة البيانات لكل صورة على حدة، مما يزيد من سرعة الحساب الإجمالية. أي نقل بيانات كبيرة دفعة واحدة أكثر كفاءة من نقل بيانات صغيرة بشكل متكرر.

■ تطبيق الدفعة في دالة التنبؤ (predict):

لتحويل الكود السابق إلى نموذج يستخدم الدفعات، يجب تغيير دالة predict لتصبح قادرة على التعامل مع مصفوفة مدخلات ذات بُعدين. ($N * 784$)

الإدخال	شكل المدخلات (صورة واحدة)	شكل المخرجات (دفعة بحجم 100)
X مدخل الطبقة الأولى	(784,)	(100,784)
W1 الأوزان	(784,50)	(784,50)
A1 الناتج قبل التفعيل	(50,)	(50,100)
Z1 الناتج بعد التفعيل	(50,)	(100,5)

الخلاصة: إن استخدام الدفعات لا يُغيّر العمليات الحسابية الأساسية داخل الشبكة (الضرب النقطي والتحيز والتفعيل)، بل يُغيّر شكل المصفوفات التي تُجرى عليها هذه العمليات، مما يُفَعِّل الاستفادة القصوى من مكتبات الحساب العددي المُحسَّنة ويسرّع عملية التدريب والاستدلال بشكل هائل. علماً أن اختيار البيانات للباتش وإرسالها يتم بالترتيب، وليس عشوائياً.

■ لنفصل العملية خطوة بخطوة:

كيف يتم اختيار البيانات للباتش (بالترتيب):

لنفترض أن لديك مجلد يحتوي على 100 صورة تريد التنبؤ بها، ولقد حددت حجم الباتش (batch size) بقيمة 32. العملية ستكون كالتالي:

الترتيب الأولي للبيانات: النظام سيقوم أولاً بإنشاء قائمة من الصور الموجودة في المجلد.

إنشاء الباتش الأول: سيأخذ النظام أول 32 صورة من القائمة (من الصورة رقم 1 إلى رقم 32) ويضعها في الباتش الأول.

إنشاء الباتش الثاني: سيأخذ النظام الـ 32 صورة التالية من القائمة (من الصورة رقم 33 إلى رقم 64) ويضعها في الباتش الثاني.

إنشاء الباتش الثالث: سيأخذ الصور من 65 إلى 96.

الباتش الأخير: سيتبقى 4 صور فقط (من 97 إلى 100). سيقوم النظام بإنشاء باتش أخير يحتوي على هذه الصور الأربع (سيكون حجمه أقل من 32).

إذن، الاختيار ليس عشوائياً على الإطلاق، بل هو اختيار متسلسل من بداية القائمة.

كيف يتم إرسال الباتشات للشبكة ؟ (بالترتيب)

بعد أن يقوم النظام بإنشاء الباتشات بالترتيب كما هو موضح أعلاه، سيقوم بإرسالها إلى الشبكة العصبونية بنفس الترتيب: يرسل الباتش الأول (الصور 1-32) للشبكة ويحصل على التنبؤات. يرسل الباتش الثاني (الصور 33-64) للشبكة ويحصل على التنبؤات. يرسل الباتش الثالث (الصور 65-96). يرسل الباتش الأخير (الصور 97-100). إذن، الإرسال أيضاً يتم بالترتيب.

لماذا الترتيب مهم في التنبؤ؟

الحفاظ على التوافق (Consistency): من المهم جداً أن تعرف أي نتيجة تنبؤ تخص أي صورة. عندما تكون العملية متسلسلة، فأنت تعرف أن التنبؤ الأول في النتائج يخص الصورة الأولى، والثاني يخص الصورة الثانية، وهكذا. لو كانت العملية عشوائية، لكان من الصعب جداً مطابقة المخرجات بالمدخلات الصحيحة.

مقارنة الدفعة مقابل المعالجة المتسلسلة (Batch vs. Sequential Processing) :

الكود الذي شرحناه سابقاً لتقييم الدقة استخدم المعالجة المتسلسلة (صورة واحدة في كل مرة)، بينما الدفعة تستخدم المعالجة المتوازية في جوهرها:

وجه المقارنة	المتسلسلة	الدفعة
شكل الإدخال	صورة واحدة في كل مرة (784,)	مجموعة من N صورة (batch-size) شكله (N,784)
عملية الحساب	تحسب مخرجات الشبكة 10000 مرة (لكل صورة اختبار)	تحسب مخرجات الشبكة مرة واحدة لكل دفعة
الكفاءة	منخفضة نسبياً لأنها تستخدم العمليات القياسية (scalar operation)	عالية جداً لأنها تستخدم العمليات الموجهة (vector/matrix operations)

وظيفة : اكتب التابع predict الذي تكلمنا عنه بالاستعانة بالتوابع المكتوبة .

حل وظيفة السابقة :

بوابة XNOR كما عالجتنا بوابة XOR سنعالج بوابة XNOR وذلك من خلال البوابات البسيطة NAND و OR ثمّ : NAND

```

20
21 #XNOR GATE
22 def XNOR(x1,x2) :
23     s1 = NAND(x1,x2)
24     s2 = OR(x1,x2)
25     y = NAND(s1,s2)
26     return y
27
28 #TEST
29 print(XNOR(0,0))
30 print(XNOR(1,0))
31 print(XNOR(0,1))
32 print(XNOR(1,1))
33

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

1
0
0
1
2
3 # NAND GATE
4 def NAND(x1,x2) :
5     w1 , w2 , theta = 0.5 , 0.5 , 0.7
6     tmp = x1 * w1 + x2 * w2
7     if tmp <= theta :
8         return 1
9     elif tmp > theta :
10        return 0
11 # OR GATE
12 def OR(x1,x2) :
13     w1 , w2 , theta = 0.5 , 0.5 , 0.3
14     tmp = x1 * w1 + x2 * w2
15     if tmp >= theta :
16         return 1
17     elif tmp < theta :
18         return 0
19

```